

Advanced Programming

Assignment Submission | Spring 2026

Name	Gaurav Nath
Roll No.	CSB24073
Subject	Advanced Programming
Assignment	Assignment No. 15
Semester	Spring 2026
Language	C (GCC + POSIX Threads)

Multithreaded Counter — Race Condition and Mutex Fix

This assignment demonstrates the race condition problem that occurs when multiple threads access a shared counter without synchronization, and then solves it using a mutex (`pthread_mutex_t`) to protect the critical section.

Question

Write a multithreaded C program using POSIX threads where multiple threads increment a shared global counter variable many times. First implement the program without any synchronization and observe the incorrect output caused by a race condition. Then modify the program using a mutex to protect the critical section and produce the correct final counter value.

The program must demonstrate:

- Thread creation using `pthread_create()`
- Synchronization using `pthread_mutex_lock()` and `pthread_mutex_unlock()`
- Thread completion using `pthread_join()`
- Why the race condition occurs and how the mutex solves the problem

Why the Race Condition Occurs

When two threads do `counter++` at the same time, that one line is actually three separate CPU operations:

```
counter++ is NOT one operation. It is THREE steps:
```

1. READ -- load counter value from memory into CPU register
2. ADD -- add 1 to the value inside the register
3. WRITE -- store the result back into memory

PROBLEM: If Thread 1 reads counter = 5
and Thread 2 ALSO reads counter = 5 before Thread 1 writes back,
both threads write 6 -- one increment is permanently lost.

This is the Race Condition.

How Mutex Solves the Problem

A mutex acts like a locked door. Only one thread can hold the lock at a time. All other threads must wait at the door until the lock is released.

```
pthread_mutex_lock(&lock);    // Thread enters -- door locked behind it
counter++;                    // Critical section -- only this thread is here
pthread_mutex_unlock(&lock);  // Thread exits -- door opens for next thread
```

Result: counter++ is now effectively atomic -- no thread can interrupt it.

Without Mutex vs With Mutex

Aspect	Without Mutex	With Mutex
Threads access counter	Simultaneously — all at once	One at a time — door locked
counter++ is atomic	No — 3 separate CPU steps	Yes — locked as one unit
Result	Random wrong value every run	Always exactly 500000
Lost updates	Thousands per run	Zero
Speed	Slightly faster	Slightly slower
Correct	No	Yes

File Structure

```
Advanced Programming 2/
└─ Assignment15/
    └─ without_mutex.c    (Version 1 -- shows the bug)
    └─ with_mutex.c      (Version 2 -- shows the fix)
```

Version 1 — without_mutex.c (Race Condition Demo)

Shows the incorrect output caused by unsynchronized thread access.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS      5
#define INCREMENT_PER_THREAD 100000

/* shared global counter -- NO protection */
int counter = 0;
```

```

void *increment(void *arg) {
    /* each thread increments counter many times */
    for (int i = 0; i < INCREMENT_PER_THREAD; i++) {
        counter++;    /* NOT atomic -- race condition happens here */
    }
    return NULL;
}

int main(void) {
    pthread_t threads[NUM_THREADS];

    printf("=== WITHOUT MUTEX (Race Condition Demo) ===\n");
    printf("Threads      : %d\n",    NUM_THREADS);
    printf("Increments    : %d per thread\n", INCREMENT_PER_THREAD);
    printf("Expected      : %d\n",    NUM_THREADS * INCREMENT_PER_THREAD);

    /* create all threads */
    for (int i = 0; i < NUM_THREADS; i++) {
        if (pthread_create(&threads[i], NULL, increment, NULL) != 0) {
            fprintf(stderr, "ERROR: pthread_create failed.\n");
            return 1;
        }
    }

    /* wait for all threads to finish */
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Actual result: %d\n", counter);
    printf("Lost updates : %d\n", (NUM_THREADS * INCREMENT_PER_THREAD) - counter);
    printf("=====\n");
    return 0;
}

```

Version 2 — with_mutex.c (Correct Output)

Shows the correct output after protecting the critical section with a mutex.

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS      5
#define INCREMENT_PER_THREAD 100000

/* shared global counter */
int counter = 0;

/* mutex to protect the critical section */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *increment(void *arg) {
    for (int i = 0; i < INCREMENT_PER_THREAD; i++) {

        pthread_mutex_lock(&lock);    /* LOCK -- only one thread enters    */
        counter++;                    /* critical section -- now safe    */
        pthread_mutex_unlock(&lock);  /* UNLOCK -- next thread can enter */
    }
    return NULL;
}

```

```

}

int main(void) {
    pthread_t threads[NUM_THREADS];

    printf("=== WITH MUTEX (Correct Output) ===\n");
    printf("Threads      : %d\n",    NUM_THREADS);
    printf("Increments    : %d per thread\n", INCREMENT_PER_THREAD);
    printf("Expected      : %d\n",    NUM_THREADS * INCREMENT_PER_THREAD);

    /* create all threads */
    for (int i = 0; i < NUM_THREADS; i++) {
        if (pthread_create(&threads[i], NULL, increment, NULL) != 0) {
            fprintf(stderr, "ERROR: pthread_create failed.\n");
            return 1;
        }
    }

    /* wait for all threads to finish */
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Actual result: %d\n", counter);
    printf("Difference   : %d\n", (NUM_THREADS * INCREMENT_PER_THREAD) - counter);
    printf("=====\n");
    return 0;
}

```

Compile and Run

```

cd "D:\Downloads D Folder\Advanced Programming 2\Assignment15"

# Compile Version 1
gcc without_mutex.c -o without_mutex -lpthread

# Run Version 1 -- observe incorrect output
.\without_mutex

# Compile Version 2
gcc with_mutex.c -o with_mutex -lpthread

# Run Version 2 -- observe correct output
.\with_mutex

```

Expected Output

Version 1 — Without Mutex (Incorrect):

```

=== WITHOUT MUTEX (Race Condition Demo) ===
Threads      : 5
Increments    : 100000 per thread
Expected      : 500000
Actual result: 387241      <- different every run, always wrong
Lost updates  : 112759
=====

```

Version 2 — With Mutex (Correct):

```
=== WITH MUTEX (Correct Output) ===
Threads      : 5
Increments   : 100000 per thread
Expected     : 500000
Actual result: 500000      <- always correct
Difference   : 0
=====
```

Key Functions Used

Function	Where	Purpose
pthread_create()	main()	Creates a new thread and starts executing the given function
pthread_join()	main()	Waits for a thread to finish before continuing
pthread_mutex_lock()	increment()	Locks the mutex — blocks other threads from entering
pthread_mutex_unlock()	increment()	Unlocks the mutex — allows next waiting thread to enter
PTHREAD_MUTEX_INITIALIZER	global	Statically initialises the mutex without calling pthread_mutex_init
-lpthread flag	gcc command	Links the POSIX thread library — required for all pthread programs

Output Screenshots

Terminal — Version 1 without mutex showing race condition (wrong result):

```
Compiled Successfully. memory: 3840 time: 0.02 exit code: 0

=== WITHOUT MUTEX (Race Condition Demo) ===
Threads      : 5
Increments   : 100000 per thread
Expected     : 500000
Actual result: 364760
Lost updates : 135240
=====
```

Terminal — Version 2 with mutex showing correct result:

```
Compiled Successfully. memory: 3584 time: 0.05 exit code: 0
```

```
=== WITH MUTEX (Correct Output) ===
```

```
Threads      : 5
```

```
Increments   : 100000 per thread
```

```
Expected     : 500000
```

```
Actual result: 500000
```

```
Difference   : 0
```

```
=====
```